

PLAN REALIZACJI

Rozgrywki Ligowe

Czysty Node.js • PostgreSQL • Angular

Kompletny plan architektury, implementacji i nauki

CEL Stworzenie pełnej aplikacji internetowej do zarządzania ligą piłkarską oraz praktyczna nauka mechanizmów HTTP w Node.js i budowy interfejsów w Angularze.

Wersja 1.0 | lipiec 2026

1. Cel i zakres projektu

Projekt będzie pełną aplikacją typu full stack. Backend zostanie zbudowany bez Expressa, NestJS i ORM, aby świadomie poznać routing, cykl żądania HTTP, walidację, obsługę błędów, transakcje i komunikację z bazą. Frontend powstanie jako osobna aplikacja Angular.

- Angular – interfejs użytkownika i prezentacja danych.
- Czysty Node.js z TypeScript – REST API i logika biznesowa.
- PostgreSQL – trwałe dane, relacje, ograniczenia i statystyki.
- Docker Compose – spójne środowisko lokalne.
- GitHub Actions – automatyczne sprawdzanie jakości i testów.

ZASADA PROJEKTOWA Angular nigdy nie łączy się bezpośrednio z PostgreSQL. Każda operacja przechodzi przez REST API backendu.

2. Funkcje aplikacji

Obszar	Zakres
Kluby	Lista, szczegóły, dodawanie, edycja, usuwanie i zawodnicy klubu.
Zawodnicy	CRUD, przypisanie do klubu, wyszukiwanie i transfer.
Sezony	CRUD, daty oraz mecze sezonu.
Mecze	Terminarz, gospodarze, goście, status i szczegóły.
Występy	Skład, rola, minuty, kartki i faule.
Bramki	Strzelec, asysta, minuta i samobój.
Statystyki	Strzelcy, kartki i liczba występów z filtrami.

3. Architektura systemu

1. Angular wysyła żądanie HTTP i dane JSON do REST API.
2. Router Node.js rozpoznaje metodę, adres i parametry żądania.
3. Kontroler przekazuje dane do serwisu domenowego.
4. Serwis stosuje walidację i reguły biznesowe.
5. Repozytorium wykonuje parametryzowane zapytania SQL.

6. PostgreSQL zwraca wynik, który wraca do Angulara jako JSON.

4. Organizacja repozytorium

Jedno repozytorium powinno zawierać niezależne katalogi backendu, frontendu, migracji i dokumentacji. Na początku nie jest potrzebny Nx ani inny system monorepo.

Katalog	Przeznaczenie
backend	Serwer Node.js, logika, zapytania SQL i testy API.
frontend	Aplikacja Angular, widoki, formularze i testy UI.
database	Migracje SQL, dane startowe i dokumentacja schematu.
docs	Wymagania, API, reguły biznesowe i wdrożenie.

5. Model danych

Encja	Najważniejsze pola	Relacje
Klub	id, nazwa, miasto, rok założenia	posiada zawodników; uczestniczy w meczach
Zawodnik	id, imię, nazwisko, pozycja, numer	należy do aktualnego klubu
Sezon	id, nazwa, daty	zawiera mecze
Mecz	id, data, status	sezon, gospodarz, gość
Uczestnictwo	rola, minuty, kartki, faule	mecz i zawodnik
Bramka	minuta, samobójcza	mecz, strzelec, asystujący

6. Reguły biznesowe

- Nazwa klubu i nazwa sezonu muszą być unikalne.
- Zawodnik może być przypisany wyłącznie do istniejącego klubu.
- Transfer do obecnego klubu jest niedozwolony.
- Gospodarz i gość meczu muszą być różnymi klubami.
- Zawodnik może mieć tylko jedno uczestnictwo w danym meczu.
- Minuta wejścia nie może być większa od minuty zejścia.
- Kartki, faule i minuty nie mogą być ujemne.

- Strzelec i asystujący muszą istnieć; nie mogą być tą samą osobą.
- Gole samobójcze nie są liczone w rankingu najlepszych strzelców.
- Usuwanie rekordów powiązanych musi mieć świadomie ustaloną politykę.

7. Projekt REST API

Publiczne endpointy otrzymują wspólny prefiks /api/v1. Nazwy zasobów są w liczbie mnogiej, a operacje biznesowe mają własne podścieżki.

Zasób	Operacje
Kluby	GET lista/szczegóły; POST; PUT; DELETE; zawodnicy klubu
Zawodnicy	GET; POST; PUT; DELETE; PATCH transfer
Sezony	GET; POST; PUT; DELETE; mecze sezonu
Mecze	GET; POST; PUT; DELETE; uczestnicy i bramki
Uczestnictwa	GET; POST; PUT; DELETE
Bramki	GET; POST; PUT; DELETE
Statystyki	najlepsi strzelcy; ranking kartek; najwięcej występów
Techniczne	health oraz dokumentacja OpenAPI

7.1. Standard odpowiedzi i błędów

- 200 – poprawny odczyt lub aktualizacja.
- 201 – utworzenie zasobu.
- 204 – poprawne usunięcie bez treści.
- 400 – błędne dane wejściowe.
- 404 – brak zasobu albo trasy.
- 409 – konflikt lub duplikat.
- 413 – zbyt duże body.
- 415 – nieobsługiwany Content-Type.
- 500 – nieoczekiwany błąd serwera.

Każdy błąd powinien zawierać stabilny kod, komunikat, status, opcjonalne błędy pól, identyfikator żądania i znacznik czasu.

8. Plan implementacji backendu

Etap	Rezultat
1. Fundament projektu	TypeScript, skrypty, ESLint, Prettier, konfiguracja środowisk i struktura katalogów.
2. Warstwa HTTP	Serwer node:http, router, parametry ścieżki i query, parser JSON, CORS, 404 i globalne błędy.
3. PostgreSQL	Docker, pula połączeń, migracje SQL, seed, transakcje i osobna baza testowa.
4. Kluby	Pełny CRUD jako moduł wzorcowy, unikalność i testy.
5. Sezony	CRUD, daty, ograniczenia i kontrola usuwania.
6. Zawodnicy	CRUD, relacja z klubem, wyszukiwanie, duplikaty i transfer.
7. Mecze	CRUD, statusy, filtrowanie, sezon oraz walidacja drużyn.
8. Uczestnictwa	Skład, czas gry, kartki, faule i blokada duplikatu.
9. Bramki	Strzelec, asysta, samobój i walidacja związku z meczem.
10. Statystyki	Agregacje SQL, filtry, limit, indeksy i testy wydajności.
11. Dokumentacja	OpenAPI, przykłady requestów, odpowiedzi i błędy.

9. Architektura backendu

Warstwa	Odpowiedzialność
Router	Rozpoznaje metodę i adres, wydobywa parametry, kieruje do kontrolera.
Middleware	CORS, logi, parsowanie JSON, limit body i przyszła autoryzacja.
Controller	Odczytuje dane żądania, wywołuje serwis i ustawia odpowiedź HTTP.
Service	Realizuje reguły biznesowe, transakcje i koordynuje operacje.
Repository	Wykonuje wyłącznie parametryzowane zapytania SQL.
Database	Pilnuje relacji, ograniczeń, indeksów i integralności.

WAŻNE SQL nie powinien znajdować się w routerach ani kontrolerach. Dzięki temu późniejsza migracja do Expressa lub NestJS nie wymusi przepisywania logiki biznesowej.

10. Plan nauki i implementacji Angulara

Etap	Zakres nauki i rezultat
1. Podstawy	Komponenty, szablony, binding, @if, @for, serwisy, DI, routing, RxJS i Signals.
2. Szkielet strony	Layout, menu, responsywność, Angular Material i strona 404.
3. Kluby	Lista, szczegóły, formularz, edycja, usuwanie i obsługa błędów.
4. Zawodnicy	Wyszukiwanie, filtr klubu, profil, formularz i transfer.
5. Sezony	Lista, formularz, szczegóły i mecze sezonu.
6. Mecze	Terminarz, formularz, status i szczegóły.
7. Zdarzenia meczu	Uczestnicy, minuty, bramki, asysty, kartki i faule.
8. Statystyki	Filtry, rankingi, tabele i wykresy.
9. Jakość UX	Spinner, komunikaty, potwierdzenia, puste stany, sortowanie i paginacja.
10. Testy	Komponenty, formularze, serwisy HTTP i scenariusze Playwright.

11. Mapa stron Angulara

Trasa	Widok
/	Panel główny
/kluby	Lista klubów
/kluby/nowy	Dodawanie klubu
/kluby/:id	Szczegóły i zawodnicy klubu
/pilkarze	Lista i wyszukiwanie zawodników
/pilkarze/:id	Profil zawodnika
/pilkarze/:id/transfer	Transfer zawodnika
/sezony	Zarządzanie sezonami
/mecze	Terminarz i wyniki
/mecze/:id	Szczegóły, skład i zdarzenia
/statystyki	Rankingi, filtry i wykresy
/404	Nieznaleziona strona

12. Kolejność prac full stack

Projekt należy rozwijać pionowymi fragmentami: baza → backend → Angular → testy. Pozwala to szybko zobaczyć działający efekt i regularnie utrwalać ten sam przepływ.

Iteracja	Temat	Rezultat
1	Fundament	HTTP, PostgreSQL, Angular layout i komunikacja.
2	Kluby	CRUD API oraz pełny interfejs klubów.
3	Zawodnicy	CRUD, relacja z klubem, profil i transfer.
4	Sezony	CRUD oraz ekran sezonu.
5	Mecze	CRUD, terminarz i szczegóły.
6	Uczestnictwo	Skład, minuty, kartki i faule.
7	Bramki	Bramki, asysty, samobójce i wynik.
8	Statystyki	Agregacje SQL, filtry, rankingi i wykresy.
9	Jakość	Walidacja, testy, responsywność i OpenAPI.
10	Wdrożenie	Docker, CI, migracje i konfiguracja produkcyjna.

13. Strategia testów

13.1. Backend

- Testy jednostkowe walidatorów, serwisów, reguł biznesowych, routera i parsera JSON.
- Testy integracyjne CRUD, kluczy obcych, unikalności, transakcji, transferu i statystyk.
- Osobna baza PostgreSQL dla testów; bez zastępowania jej SQLite.

13.2. Angular

- Testy komponentów, formularzy, walidatorów i serwisów HTTP.
- Testy stanów ładowania, pustych danych i błędów API.
- Testy end-to-end najważniejszych ścieżek użytkownika.

13.3. Krytyczne scenariusze e2e

7. Utworzenie klubu i zawodnika.

8. Utworzenie sezonu i zaplanowanie meczu.
9. Dodanie składu oraz bramki.
10. Transfer zawodnika.
11. Wyświetlenie poprawnych statystyk.
12. Odrzucenie niepoprawnych danych i duplikatów.

14. Bezpieczeństwo

- Wyłącznie parametryzowane zapytania SQL.
- Limit rozmiaru body i walidacja Content-Type.
- CORS ograniczony do znanych adresów frontendu.
- Brak szczegółów błędów bazy w odpowiedziach.
- Hasła i konfiguracja tylko w zmiennych środowiskowych.
- Plik .env wykluczony z repozytorium.
- Aktualizowanie zależności i kontrola podatności.
- Poprawne zamykanie serwera i puli połączeń.

Logowanie i role warto dodać dopiero po zakończeniu podstawowej wersji domenowej. Jeżeli powstaną konta, hasła muszą być hashowane sprawdzoną biblioteką, a operacje zapisu chronione rolą administratora.

15. Docker, środowiska i CI

Element	Plan
Usługi	frontend, backend i postgres
Środowiska	development, test i production
Konfiguracja	osobne bazy, adresy API i poziomy logowania
CI	formatowanie, lint, TypeScript, testy backendu i Angulara, build
Migracje	wykonywane kontrolowanie przed uruchomieniem nowej wersji

16. Git i organizacja pracy

- Gałąź main powinna zawsze zawierać stabilną wersję.
- Każda funkcja powstaje na krótkiej gałęzi feature.
- Pull request powinien obejmować jeden spójny zakres.

- Commity powinny być małe i opisywać konkretną zmianę.
- Zmian w schemacie bazy nie wolno wprowadzać bez migracji.

17. Harmonogram edukacyjny

Tydzień	Główny zakres
1	Node.js, TypeScript, HTTP i routing
2	PostgreSQL, SQL, migracje i klub
3	Podstawy Angulara i interfejs klubów
4	Zawodnicy, relacje i transfer
5	Sezony i mecze
6	Uczestnictwo i czas gry
7	Bramki, kartki i faule
8	Statystyki i wykresy
9	Testy oraz poprawa interfejsu
10	Docker, CI, dokumentacja i wdrożenie

SZACUNEK Przy równoległej nauce Angulara i Node.js realistyczny czas realizacji wynosi około 8–14 tygodni, zależnie od liczby godzin tygodniowo.

18. Zakres pierwszej wersji

18.1. Wymagane

- CRUD klubów, zawodników, sezonów i meczów.
- Uczestnicy meczu, minuty, kartki i faule.
- Bramki, asysty oraz samobóje.
- Transfer zawodnika.
- Trzy rankingi statystyczne.
- Responsywna strona Angular.
- PostgreSQL, migracje i Docker Compose.
- Walidacja, testy i dokumentacja uruchomienia.

18.2. Odłożone na później

- Logowanie i role użytkowników.
- Zdjęcia i upload plików.
- Powiadomienia oraz WebSocket.
- NgRx i mikroserwisy.
- Aplikacja mobilna.
- Zaawansowane raporty i eksporty.

19. Pierwszy kamień milowy

Pierwszy działający fragment powinien objąć cały przepływ dla klubów. Będzie wzorcem dla wszystkich kolejnych modułów.

13. PostgreSQL uruchomiony w Dockerze.
14. Migracja tabeli klubów.
15. Serwer HTTP Node.js.
16. Pełny CRUD klubów.
17. Jednolity format błędów.
18. Angular z listą klubów.
19. Formularz dodawania i edycji.
20. Usuwanie z potwierdzeniem.
21. Testy pełnego przepływu.

20. Kryteria ukończenia projektu

- Wszystkie zaplanowane funkcje działają przez interfejs Angular.
- API ma spójne endpointy, statusy HTTP i błędy.
- Reguły biznesowe są egzekwowane w backendzie i bazie.
- Statystyki zwracają poprawne wyniki dla danych testowych.
- Migracje tworzą bazę od zera.
- Testy jednostkowe, integracyjne i e2e przechodzą w CI.
- Cały system uruchamia się udokumentowanym procesem.
- Sekrety nie znajdują się w repozytorium.
- Interfejs jest czytelny na komputerze i urządzeniu mobilnym.

21. Dalszy rozwój backendu

Po ukończeniu wersji edukacyjnej można pozostać przy czystym Node.js albo przenieść warstwę HTTP do Expressa lub NestJS. Podział na router, controller, service i repository ograniczy zakres migracji. Najczęściej wymienione zostaną router, middleware i konfiguracja, natomiast serwisy oraz reguły biznesowe powinny pozostać prawie bez zmian.

REKOMENDACJA KOŃCOWA Rozwijaj system pionowo. Najpierw ukończ Kluby od bazy po Angular i testy, a następnie powtarzaj ten wzorzec dla kolejnych modułów. Nie próbuj pisać całego backendu przed rozpoczęciem interfejsu.